MAVERIC - Perception

Autonomous Vehicle Sensor Fusion & Object Tracking Stack

Akshat G
Computer Vision Department - CEAM

February 27, 2026

Contents

1 Introduction: The MAVERIC Perception Stack **2**

2 System Pipeline Overview **2**

3 Semantic Segmentation Optimization (SegFormer) **2**
3.1 Exporting PyTorch to ONNX . . . 2
3.2 Compiling the TensorRT Engine . . . 3

4 Spatial Coordinate Transformation **4**
4.1 Transformation Sequence . . . 4

5 LiDAR Point Painting and Sensor Alignment **5**
5.1 Coordinate Transformations . . . 5
5.2 Pinhole Projection and Mask Querying . . . 5

6 2D Object Detection Optimization (YOLOv8) **6**
6.1 TensorRT Compilation . . . 6
6.2 Inference and Bounding Box Extraction . . . 6

7 Pre-Processing: Heuristic Ground Removal **7**

8 Double-Filter Frustum Depth Fusion **7**
8.1 Filter 1: Spatial Cropping (The Frustum) . . . 7
8.2 Filter 2: Semantic Verification . . . 7
8.3 Depth Calculation and Proportionality Heuristic . . . 8

Implementation Notes and Future Work **9**

9 Hardware Synchronization & Collection **9**
9.1 Temporal Synchronization . . . 9
9.2 Global Positioning (Ego Pose) . . . 9

10 Relational Schema Architecture **9**
10.1 `sensor.json` . . . 9
10.2 `calibrated_sensor.json` . . . 10
10.3 `ego_pose.json` . . . 10
10.4 `sample.json` . . . 10
10.5 `sample_data.json` . . . 10
10.6 Relational Join Interpretation . . . 11

11 Calibration Methodologies **11**

12 Reimplementation Pipeline **11**

Abstract

This document describes the early design and implementation plan for the MAVERIC perception stack prototype. It acts as a working technical record, capturing the system structure, key assumptions, and practical engineering decisions that shape the initial version of the pipeline. The sections walk through the perception flow from sensor inputs and calibration, through semantic interpretation, depth estimation, and fusion logic, while also outlining how these components are expected to interact in a real deployment.

Rather than serving as final implementation documentation, this document functions as a foundation and reference point for the prototype phase. A separate, more detailed implementation report will follow as the system matures and design choices stabilize.

1 Introduction: The MAVERIC Perception Stack

The fundamental challenge of autonomous perception is extracting reliable, 3D geometric understanding from a dynamic environment in real-time. The MAVERIC perception stack is designed to operate at ≈ 10 Hz, perfectly matching the sweep frequency of standard automotive LiDAR sensors.

To achieve this without relying on computationally prohibitive 3D deep learning models, the MAVERIC architecture heavily utilizes early-stage hardware acceleration (NVIDIA TensorRT) combined with deterministic geometric physics. This document chronicles the step-by-step construction of this pipeline, beginning with the foundational layer: Semantic Understanding.

2 System Pipeline Overview

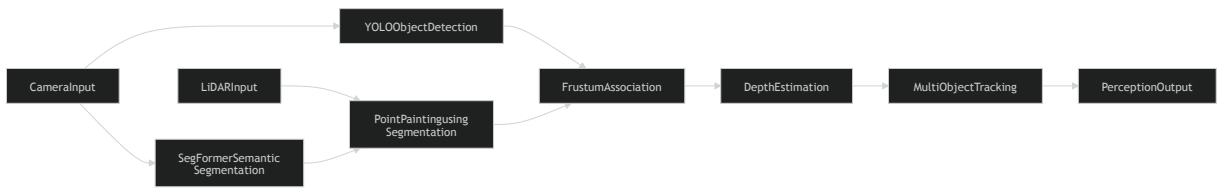

Figure 1: High-level MAVERIC perception pipeline overview.

3 Semantic Segmentation Optimization (SegFormer)

Before the system can reason about distance or velocity, it must understand the contextual identity of its surroundings. We require a model capable of densely classifying every pixel in a high-resolution camera feed into structural (roads, buildings, sidewalks) and dynamic (cars, pedestrians, bicycles) categories.

For the MAVERIC platform, we selected NVIDIA's `SegFormer-B1` architecture. Unlike traditional Convolutional Neural Networks (CNNs), SegFormer utilizes a hierarchically structured Transformer encoder combined with a lightweight Multi-Layer Perceptron (MLP) decoder. This design explicitly bypasses complex positional encodings, making it remarkably efficient for high-resolution driving scenes. The model utilized is fine-tuned on the Cityscapes dataset, natively outputting 19 distinct urban classes at a 1024×1024 resolution.

However, executing raw PyTorch transformer models incurs significant Python overhead and inefficient memory allocation, rendering it entirely unsuited for real-time vehicular deployment. To bridge the gap between research and production, a strict optimization pipeline to TensorRT was executed.

3.1 Exporting PyTorch to ONNX

The first crucial step in the deployment pipeline is decoupling the model from the PyTorch runtime. We achieve this by tracing the PyTorch computation graph and exporting it to the Open Neural Network Exchange (ONNX) format.

ONNX serves as an intermediary, framework-agnostic representation of the model's mathematical operations. To generate this graph, we must supply the model with a "dummy tensor." This tensor does not contain real image data; it exists solely to define the static input dimensions ($1 \times 3 \times 1024 \times 1024$) so the tracing compiler understands the exact memory footprint required for inference.

```python
import torch
from transformers import SegformerForSemanticSegmentation

# 1. Load Pretrained HuggingFace Model (B1 variant for optimal speed/accuracy)
model_name = "nvidia/segformer-b1-finetuned-cityscapes-1024-1024"
model = SegformerForSemanticSegmentation.from_pretrained(model_name)
model.eval().cuda() # Lock weights and move to GPU

# 2. Define static dummy input (Batch=1, Channels=3, H=1024, W=1024)
dummy_input = torch.randn(1, 3, 1024, 1024, device='cuda')

# 3. Export to ONNX representation
torch.onnx.export(
    model,
    dummy_input,
    "segformer_b1.onnx",
    opset_version=13,       # Opset 13 ensures compatibility with modern TensorRT versions
    input_names=['input'],  # Explicitly name the binding nodes
    output_names=['output']
)
print("ONNX Export Successful: segformer_b1.onnx generated.")
```

Listing 1: Generating the ONNX Computation Graph

Setting the `opset_version=13` is critical, as it ensures that the specific transformer operations (like LayerNorm and GeLU activations) are mathematically supported by the subsequent TensorRT compiler.

3.2 Compiling the TensorRT Engine

With the static `.onnx` graph generated, the model is ready for hardware-specific compilation. We utilize NVIDIA's `trtexec` command-line utility to convert the ONNX graph into a highly optimized `.engine` file.

This compilation phase performs several advanced optimizations:

- **Layer & Tensor Fusion:** Vertically fusing sequential operations (e.g., Convolution + BatchNorm + ReLU) into a single GPU kernel execution to minimize memory bandwidth bottlenecks.
- **Kernel Auto-Tuning:** Benchmarking various underlying CUDA algorithms against the specific GPU silicon (e.g., matching the optimal matrix multiplication tactic to the target architecture).
- **Precision Calibration (FP16):** Quantizing the model's 32-bit floating-point weights down to 16-bit. This instantly halves the memory footprint and significantly accelerates inference on modern Tensor Cores, with a practically invisible loss in segmentation accuracy.

```bash
$ trtexec --onnx=segformer_b1.onnx \
          --saveEngine=segformer_b1_python_build.engine \
          --fp16 \
          --workspace=4096
```

Listing 2: Hardware Compilation via trtexec

In the execution above, the `--workspace=4096` flag allocates 4 Gigabytes of temporary VRAM. The TensorRT compiler uses this workspace as a "scratchpad" to test and profile different layer fusion strategies before settling on the fastest possible execution plan.

The resulting `segformer_b1_python_build.engine` file allows the MAVERIC perception stack to ingest high-resolution camera frames and output dense semantic masks in under 15 milliseconds, safely within the 100ms budget required for 10 Hz real-time operation.

4 Spatial Coordinate Transformation

Fusing data from unsynchronized, physically displaced sensors on a moving ego-vehicle requires a rigorous multi-step rigid body transformation. A LiDAR sweep and a camera capture occur at slightly different timestamps (t_{lidar} and t_{cam}). To correctly map a 3D LiDAR point $P_{lidar} = [X, Y, Z, 1]^T$ to a 2D camera pixel $[u, v, 1]^T$, we route the point through the global map frame.

4.1 Transformation Sequence

1. **LiDAR to Ego Frame (at t_{lidar}):**

$$P_{ego_lidar} = R_{lidar_ext} P_{lidar} + T_{lidar_ext} \tag{1}$$

2. **E Ego Frame to Global Map (at t_{lidar}):**

$$P_{global} = R_{ego_pose(t_lidar)} P_{ego_lidar} + T_{ego_pose(t_lidar)} \tag{2}$$

3. **Global Map to Ego Frame (at t_{cam}):**

$$P_{ego_cam} = R^{-1}_{ego_pose(t_cam)} (P_{global} - T_{ego_pose(t_cam)}) \tag{3}$$

4. **Ego Frame to Camera Sensor (at t_{cam}):**

$$P_{cam} = R^{-1}_{cam_ext} (P_{ego_cam} - T_{cam_ext}) \tag{4}$$

5. **Camera Coordinates to 2D Image Pixels:**

$$Z_c \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K_{cam} P_{cam} \tag{5}$$

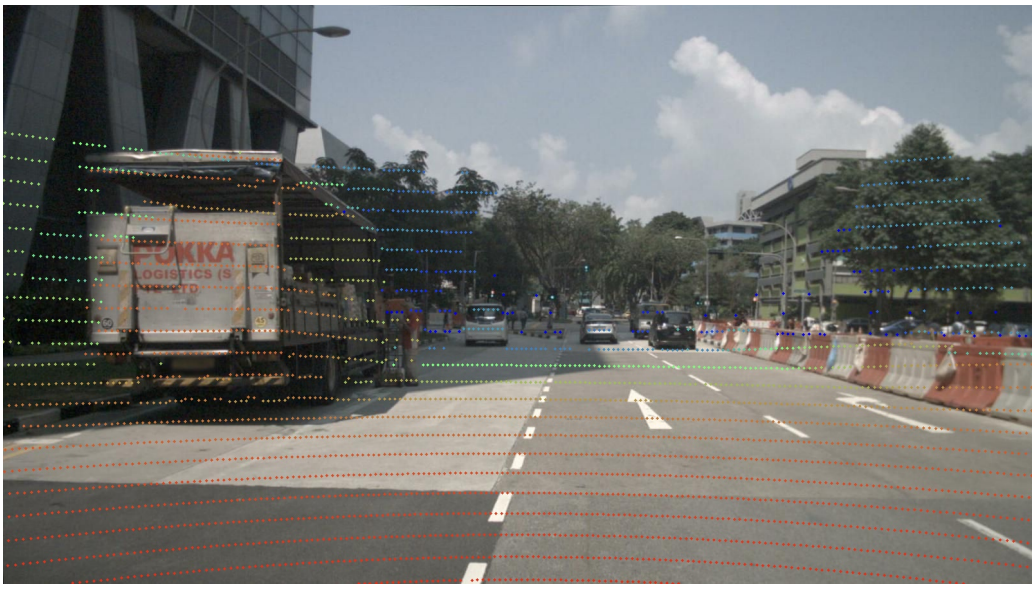

Figure 2: Projection of 3D LiDAR points onto 2D CAM_FRONT image space.

5 LiDAR Point Painting and Sensor Alignment

With the semantic mask successfully generated by the TensorRT-optimized SegFormer engine, the next critical phase is *Point Painting*. Raw LiDAR point clouds provide highly accurate depth (Z) and structural (X, Y) information but entirely lack dense contextual meaning. Point Painting resolves this by projecting the 3D LiDAR points onto the 2D camera plane and appending the semantic class of the intersecting pixel.

5.1 Coordinate Transformations

To project a 3D LiDAR point onto a 2D camera image, it must traverse multiple coordinate frames. Using the calibration parameters extracted from the system's JSON metadata (kindly refer to section 10 for the JSON metadata format). The transformation sequence has been talked about in greater detail in previous section section 4, but an overview of the process can be infered as follows:

1. **LiDAR to Ego-Vehicle:** Apply the LiDAR's extrinsic rotation and translation to move the point into the vehicle's central coordinate frame.

2. **Ego-Vehicle to Global Map:** Apply the ego-vehicle's pose to anchor the point in the global world frame.

3. **Global to Camera-Ego:** Reverse the ego-pose at the camera's specific timestamp to account for any micro-movements between sensor sweeps.

4. **Camera-Ego to Camera Sensor:** Apply the inverse of the camera's extrinsic matrix to move the point into the local 3D camera frame.

5.2 Pinhole Projection and Mask Querying

Once the points are successfully transformed into the local 3D camera frame, we apply the camera's intrinsic matrix (K) to perform the pinhole projection, mathematically mapping the 3D coordinates $[X, Y, Z]$ to 2D image pixels $[u, v]$ as seen in figure Figure 2.

Points that fall outside the image boundaries ($0 \leq u <$ Width, $0 \leq v <$ Height) or have a depth $Z < 0.1$m (located behind the camera lens) are systematically discarded. For every valid projected point, we query the 1024×1024 SegFormer output mask at coordinate (u, v). The resulting semantic class ID is appended to the LiDAR point, effectively "painting" the structural geometry with contextual intelligence as shown in Figure 3.

Figure 3: LiDAR points successfully projected onto the camera frame and painted with SegFormer semantics.

6 2D Object Detection Optimization (YOLOv8)

While semantic segmentation provides dense pixel-level context, defining discrete object boundaries (instances) purely from segmentation blobs is highly susceptible to noise, partial occlusions, and overlapping instances. To resolve this, the MAVERIC pipeline employs YOLOv8 (Nano variant) to generate robust, jitter-free 2D geometric proposals. Note that initially a density based clutering (DBSCAN) was implemented on the LiDAR. Due to LiDAR sampling dynamics and the overall difficulty in trying to implement a good clustering algorithm, the DBSCAN approach was dropped. Furtermore, DBSCAN inferred around 20ms to 40ms at 95 percentile, and hence was concluded that the variance in clustering was too high and a traditional camera based approach could be taken. More improvements to this phase of the pipeline are undoubtedly necessary.

6.1 TensorRT Compilation

Similar to the SegFormer pipeline, raw PyTorch execution of YOLOv8 is too slow for real-time 10 Hz constraints. We utilize the native `ultralytics` backend to export the model through ONNX directly into a Half-Precision (FP16) TensorRT engine.

```python
from ultralytics import YOLO

# 1. Load pretrained PyTorch YOLOv8 Nano
model = YOLO("yolov8n.pt")

# 2. Export directly to TensorRT engine (FP16 quantization)
model.export(
    format="engine",
    half=True,
    imgsz=640,
    workspace=4
)
print("Engine exported successfully as yolov8n.engine")
```

Listing 3: Exporting YOLOv8 to an FP16 TensorRT Engine

This compilation fuses convolutional layers and quantizes weights to 16-bit, achieving an inference time of $\approx 5-8$ milliseconds per frame on the target hardware.

6.2 Inference and Bounding Box Extraction

During the runtime loop, the TensorRT engine ingests the 1600×900 camera frame and outputs coordinate proposals. We explicitly map the YOLO/COCO class IDs to our centralized Cityscapes IDs to maintain a unified taxonomy across the perception stack.

```python
# Run TensorRT Inference (Suppressing CLI output for speed)
results = yolo_engine(image, verbose=False)[0]

# Extract raw bounding box coordinates [x1, y1, x2, y2] and Class IDs
yolo_boxes = results.boxes.xyxy.cpu().numpy()
yolo_classes = results.boxes.cls.cpu().numpy()
```

Listing 4: Extracting 2D Proposals from the TRT Engine

7 Pre-Processing: Heuristic Ground Removal

Before routing the 3D LiDAR points into the Frustum Fusion logic (section 8), a critical pre-processing step is required to stabilize depth measurements: Ground Removal.

Because bounding boxes generated by 2D object detectors (like YOLO) typically encapsulate the contact patch where the vehicle's tires meet the road, projecting unfiltered LiDAR points into this box will invariably include returns from the asphalt. If these road points are factored into the depth calculation, the median distance estimate is artificially pulled closer to the camera, compromising the spatial accuracy of the entire pipeline.

Because the prototyping dataset operates in predominantly flat, urban environments (Boston and Singapore) and the LiDAR sensor height is fixed, we implement a highly efficient Static Z-Thresholding heuristic rather than computationally expensive RANSAC plane fitting. Note, for future implementations and compute power, a more robust RANSAC plane fitting method might be preferred.

The LiDAR sensor is mounted atop the ego-vehicle at $Z = 0.0$m. The road surface rests at approximately $Z = -1.8$m. Prior to any coordinate projection, we strip all points that fall below $Z = -1.4$m in the local Ego frame. This single operation instantly removes the vast majority of the road plane in $O(N)$ time, ensuring that only points striking physical object chassis survive to the projection phase.

```python
def remove_ground(points):
    """
    Strips points below the vehicle chassis to prevent
    depth-snapping to the road surface inside YOLO bounding boxes.
    """
    # Keep only points where Z is greater than -1.4 meters
    non_ground_mask = points[:, 2] > -1.4

    return points[non_ground_mask]

# Apply to raw LiDAR before camera projection
filtered_lidar = remove_ground(raw_lidar)
```

Listing 5: Static Z-Threshold Ground Removal

8 Double-Filter Frustum Depth Fusion

The core limitation of a 2D bounding box is its lack of depth (Z). A naive approach to finding object distance is to take all LiDAR points that project inside the 2D bounding box and calculate their average depth. However, this fails catastrophically in the real world: a bounding box drawn around a car will inevitably encapsulate the road beneath it, the trees behind it, and the sky above it. Naively averaging these LiDAR returns will calculate the distance to the background rather than the object.

To solve this, the MAVERIC pipeline introduces a **Double-Filter Frustum mechanism**, leveraging both the YOLO geometry and the SegFormer semantics to perfectly isolate the LiDAR points belonging to the physical target.

8.1 Filter 1: Spatial Cropping (The Frustum)

First, we iterate over every 2D bounding box generated by YOLO. We query the projected LiDAR points (u, v pixels) and discard any point that does not fall strictly within the $[x_{min}, y_{min}, x_{max}, y_{max}]$ boundary of the proposal. This creates a 3D "frustum" (a cone of view) extending from the camera lens outward, containing the object and its background.

8.2 Filter 2: Semantic Verification

Next, we apply the semantic filter. For every LiDAR point that survived the spatial crop, we look up its exact (u, v) pixel coordinate in the dense SegFormer mask. We compare the SegFormer class prediction against the YOLO class prediction.

If YOLO proposes a “Car”, but the SegFormer mask indicates that a specific LiDAR point within the box hit a “Tree” or the “Road”, that point is instantly deleted. *Only points that are geometrically inside the bounding box AND semantically verified as the correct class survive.*

```
for box, cls_id in zip(yolo_boxes, yolo_classes):
    x1, y1, x2, y2 = box

    # Filter 1: Is the LiDAR point inside the 2D YOLO Box?
    in_box = (depth_mask) & (pixels[:, 0] >= x1) & (pixels[:, 0] <= x2) & \
                            (pixels[:, 1] >= y1) & (pixels[:, 1] <= y2)
    pts_in_box = np.where(in_box)[0]

    if len(pts_in_box) > 0:
        # Filter 2: Does the SegFormer mask agree with the YOLO class?
        u_ins = pixels[pts_in_box, 0].astype(int)
        v_ins = pixels[pts_in_box, 1].astype(int)

        # Query the semantic mask
        semantic_labels = seg_mask[v_ins, u_ins]

        # Keep only the points where Semantics == YOLO Class
        valid_pts = pts_in_box[semantic_labels == yolo_class_id]

        if len(valid_pts) > 0:
            # Calculate final physical distance
            true_distance = np.median(depths[valid_pts])
```

Listing 6: The Double-Filter Logic for LiDAR Isolation

8.3 Depth Calculation and Proportionality Heuristic

With the background perfectly stripped away, the surviving points represent the physical surface of the target object. We calculate the median depth (Z) of these points to establish the exact distance to the object.

To ensure mathematical stability, especially at extreme ranges where LiDAR sparsity is high, we implement a dynamic point threshold: $Required_Points = \max(1, \lfloor 15.0/Z \rfloor)$. A vehicle at 5 meters requires at least 3 verified LiDAR hits to be considered valid, while a vehicle at 40 meters requires only 1.

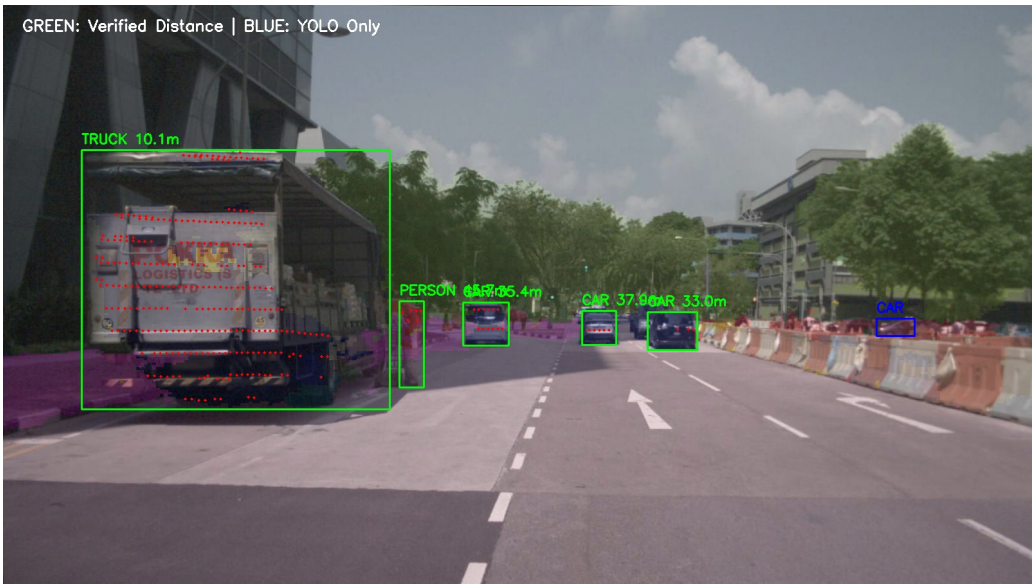

Figure 4: Successful Frustum Depth Fusion. The algorithm calculates precise depth by relying only on double-verified surface points.

Implementation Notes and Future Work

Note to the reader: The sections that follow are not part of the operational perception pipeline. They are included as implementation documentation and design references for future development.

---

9 Hardware Synchronization & Collection

This section talks about how to ensure that we can perform the required math for section 4. To accurately project LiDAR point clouds onto dynamic 2D camera images, physical hardware synchronization is non-negotiable.

9.1 Temporal Synchronization

All sensors must share a unified system clock.

- **PTP (Precision Time Protocol):** Utilize IEEE 1588 PTP over the vehicle's ethernet backbone to synchronize the LiDAR unit(s) and the computational host to sub-microsecond accuracy.
- **Hardware Triggering:** The LiDAR's top-dead-center (TDC) firing phase should output an electrical pulse (PPS/GPIO trigger) to the camera suite, ensuring the shutter fires at the exact moment the LiDAR laser sweeps across the camera's Field of View (FoV).

9.2 Global Positioning (Ego Pose)

The vehicle must record its global position and orientation at high frequencies (e.g., 100Hz) using an RTK-GPS coupled with an active IMU. This allows the pipeline to interpolate the exact X, Y, Z and Quaternion rotation of the ego-vehicle at any given microsecond.

10 Relational Schema Architecture

Rather than storing perception data as loosely organized frame folders, the MAVERIC pipeline adopts a relational metadata architecture. Each sensor measurement, calibration record, and vehicle pose is represented as a uniquely indexed entity within a structured JSON database. This design enables deterministic joins between spatial calibration, ego-motion, and raw sensor payloads, ensuring that every observation can be reconstructed in its exact geometric and temporal context.

The schema is composed of five core tables, each identified by a unique 32-character hexadecimal token. Together, they form a graph structure that links raw sensor files to their calibration parameters and the vehicle state at capture time.

10.1 `sensor.json`

This table defines the static inventory of sensing hardware mounted on the vehicle. Each entry describes the modality and logical channel name of a sensor, independent of its pose or calibration. It functions as the root taxonomy for all downstream data associations.

```
{
    "token": "725903f5b62f5611...",
    "channel": "CAM_FRONT",
    "modality": "camera"
}
```

10.2 calibrated_sensor.json

This table stores the geometric calibration of each sensor relative to the ego-vehicle coordinate frame. It includes both the rigid-body extrinsics and, where applicable, the intrinsic camera matrix. Translations are expressed in meters $[X, Y, Z]$, and rotations are normalized quaternions $[w, x, y, z]$.

```
{
    "token": "f4d2a6c281f34a7e...",
    "sensor_token": "725903f5b62f5611...",
    "translation": [3.412, 0.0, 0.5],
    "rotation": [0.999, 0.0, 0.0, 0.001],
    "camera_intrinsic": [
        [800.0, 0.0, 640.0],
        [0.0, 800.0, 360.0],
        [0.0, 0.0, 1.0]
    ]
}
```

10.3 ego_pose.json

This table represents the continuous trajectory of the vehicle in the global map frame. Each entry records the timestamped position and orientation of the ego-platform derived from the fused RTK-GPS and IMU solution. These records allow the system to interpolate the vehicle pose at arbitrary timestamps during projection and motion compensation.

```
{
    "token": "5ace90b379af485b...",
    "timestamp": 1532402927814384,
    "translation": [410.77, 1179.46, 0.0],
    "rotation": [0.573, -0.001, 0.013, -0.819]
}
```

10.4 sample.json

A *sample* represents a synchronized perception key-frame. It serves as the temporal anchor for multi-sensor fusion and contains references to the closest sensor measurements captured at that timestamp. The dictionary structure allows flexible association of heterogeneous sensors without enforcing rigid frame hierarchies.

```
{
    "token": "ca9a282c9e77460f...",
    "timestamp": 1532402927647951,
    "data": {
        "CAM_FRONT": "data_token_1...",
        "LIDAR_TOP": "data_token_2..."
    }
}
```

10.5 sample_data.json

This table links logical measurements to their physical storage locations. Each record specifies the file path of the sensor payload and references the exact calibration and ego-pose valid at the capture timestamp. In relational terms, this table functions as the central bridge connecting raw data, calibration metadata, and vehicle state.

```
{
    "token": "data_token_1...",
    "sample_token": "ca9a282c9e77460f...",
    "calibrated_sensor_token": "f4d2a6c281f34a7e...",
    "ego_pose_token": "5ace90b379af485b...",
    "filename": "sweeps/CAM_FRONT/image_001.jpg",
    "timestamp": 1532402927814384
}
```

10.6 Relational Join Interpretation

When reconstructing a scene, the processing pipeline performs SQL-style joins across these tables:

$$\texttt{sample} \rightarrow \texttt{sample_data} \rightarrow \{\texttt{calibrated_sensor}, \texttt{ego_pose}\}$$

This guarantees that every sensor observation is associated with the correct geometric calibration and vehicle pose, enabling reproducible spatial transformations and temporally consistent sensor fusion.

11 Calibration Methodologies

To populate the `calibrated_sensor.json` table, strict physical calibration routines must be executed prior to data collection.

1. **Intrinsic Calibration:** A standard checkerboard must be passed through the entire field of view of each camera to extract the 3×3 intrinsic matrix K and distortion coefficients D. Images should be undistorted prior to being saved to the dataset.

2. **Extrinsic Calibration:** Precise measurements from the vehicle's rear axle center (standard ego-frame origin) to the optical center of each sensor. Final fine-tuning is achieved via software-based Targetless Hand-Eye Calibration, minimizing the projection error between LiDAR edge geometries and camera pixel gradients.

12 Reimplementation Pipeline

To process data formatted under this protocol:

1. Parse the JSONs using an SQL-style join architecture mapping `sample` $\rightarrow$ `sample_data` $\rightarrow$ `calibrated_sensor` & `ego_pose`.

2. Route 3D points through the rigid body transformations: LiDAR $\rightarrow$ Ego $\rightarrow$ Global $\rightarrow$ Ego $\rightarrow$ Camera.

3. Extract semantic segmentations and append as one-hot encoded arrays to generate painted `.bin` point clouds.